香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

ByteDance

*ACE: Sending **Burstiness** Control for High Quality Real-Time Communication*

Xiangjie Huang, Jiayang Xu, Haiping Wang, Hebin Yu
Sandesh Dhawaskar Sathyanarayana, Shu Shi, Zili Meng

March 18th, 2026

Background
What is RTC, and why is it hard?

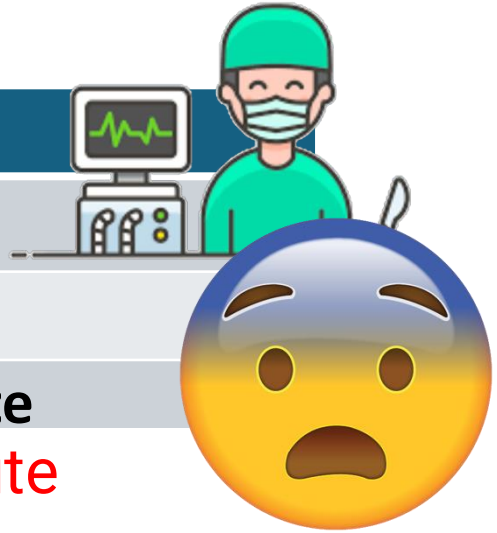
Real-Time Communication (RTC) is indispensable in lives.

*Slide taken from Salsify [NSDI'18].

Background
Frustrating Latency Issues

Tail Latency issues block the path of future applications.

Paper	Metric	Value
Zhuge [Sigcomm'22]	99th Percentile Latency	~400 ms
AFR [NSDI'23]	Stall rate on Wi-Fi	2.11%
Tooth [NSDI'25]	Stall frequency	**1 stall / minute**
Tooth [NSDI'25]	Stall frequency	1 stall / minute

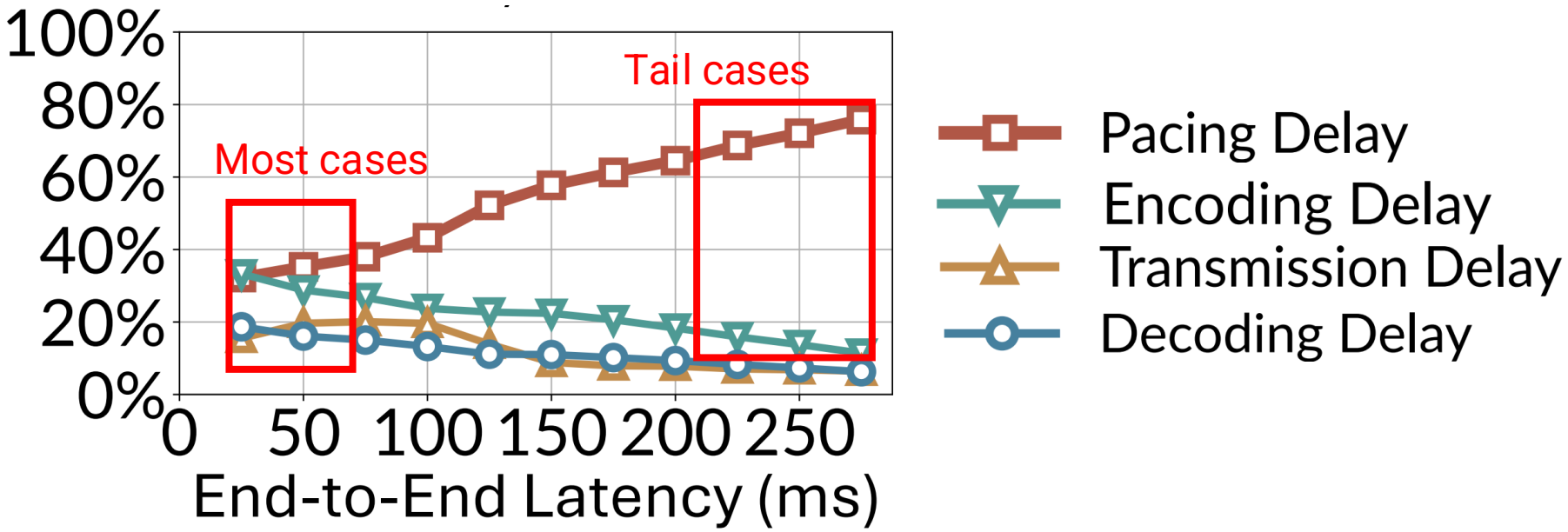


Measurements on large-scale RTC applications still show high tail latency.

Motivation
Dramatic Pacing Latency

Pacing latency can be a significant contributor to tail latency and stalls.

Evaluation on WebRTC

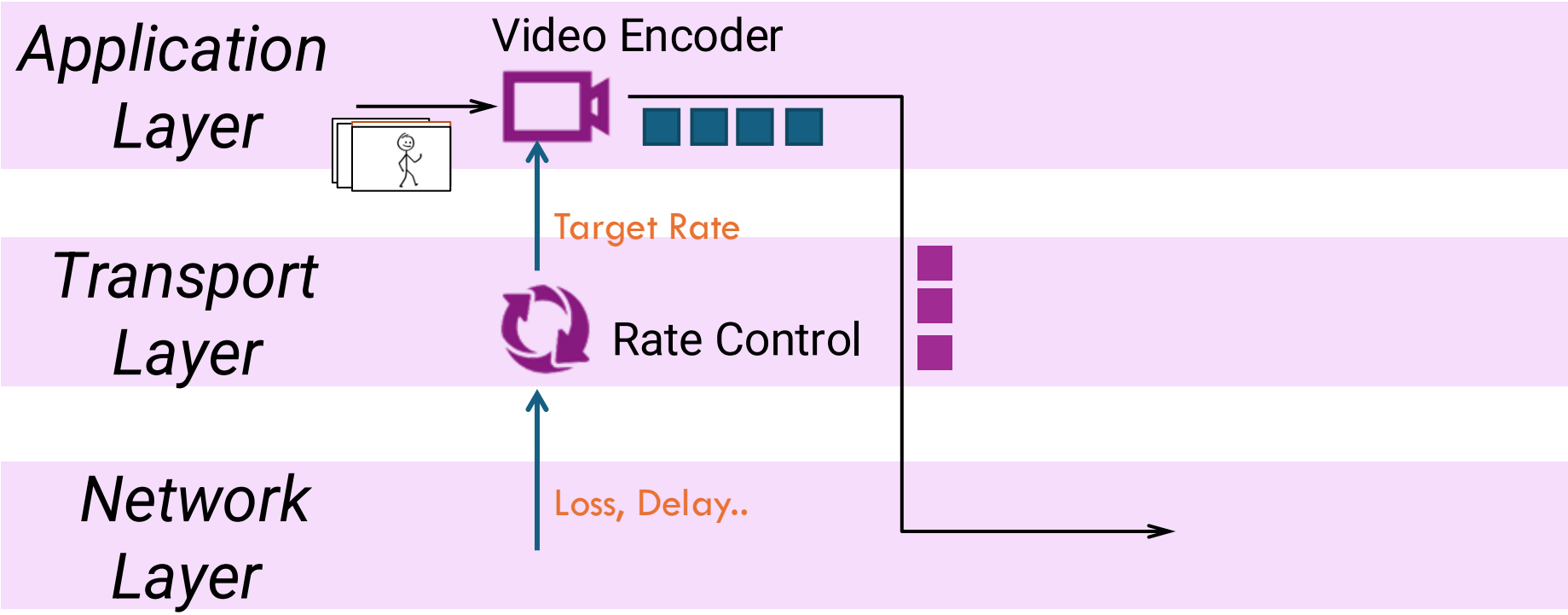


- Other components contribute less than 1/3.
- **Validation:**

From online data of Douyin cloud gaming, pacing contributes to 100 ms stalls!

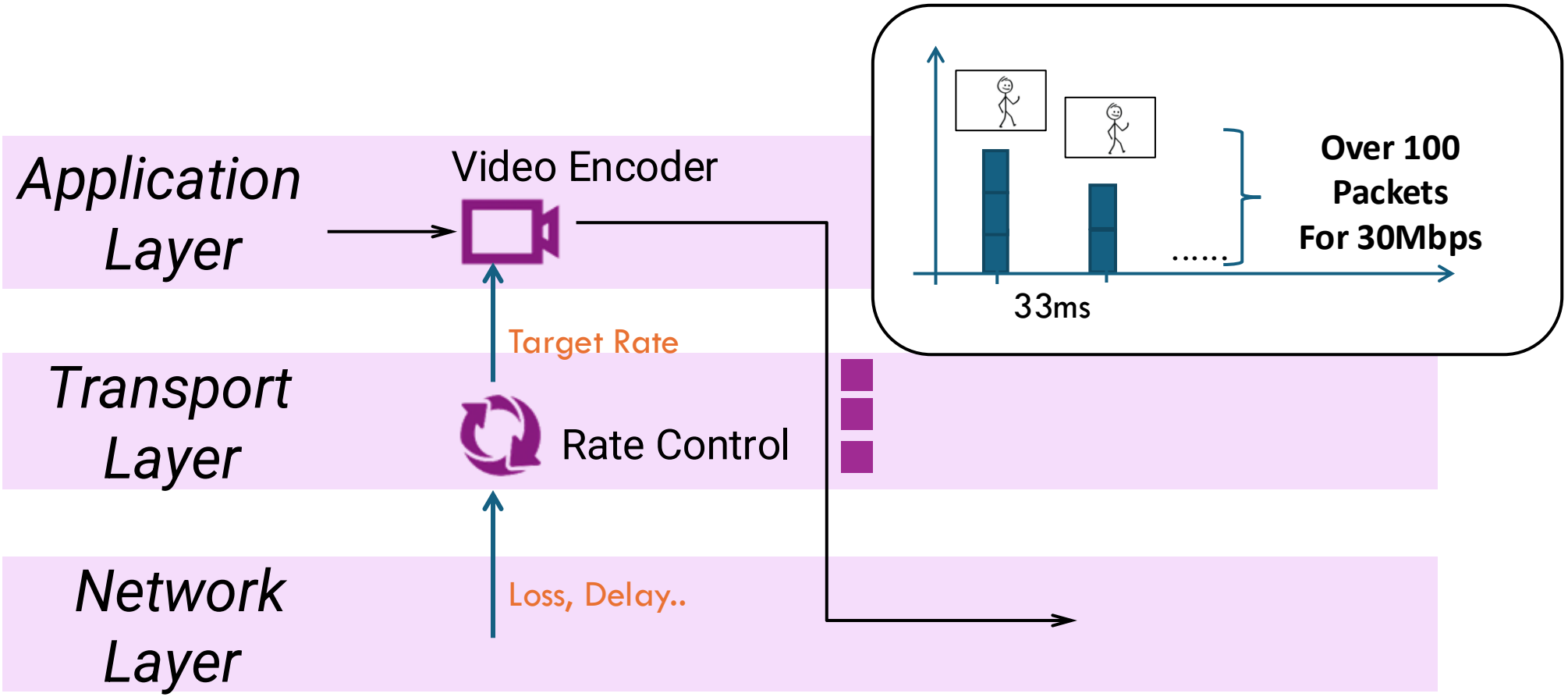
Motivation
Sending Burstiness

Sending pattern is **bursty** because of the nature of video coding.



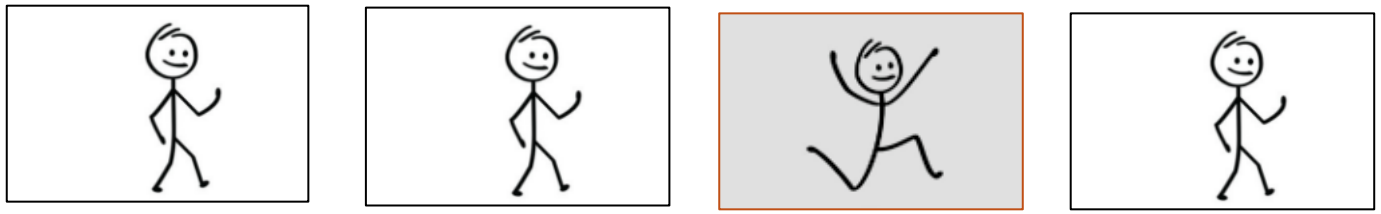
Motivation
Sending Burstiness

Sending pattern is **bursty** because of the nature of video coding.

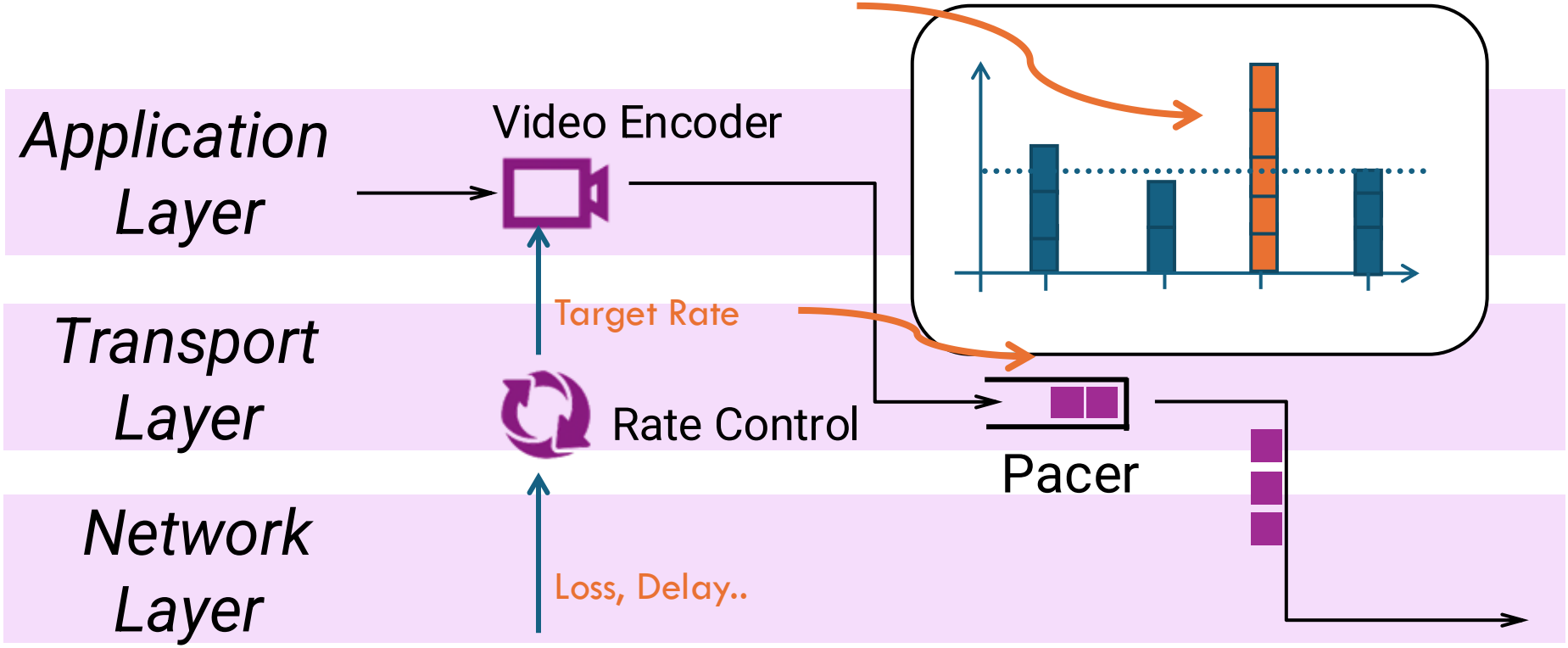


Motivation
Sending Burstiness

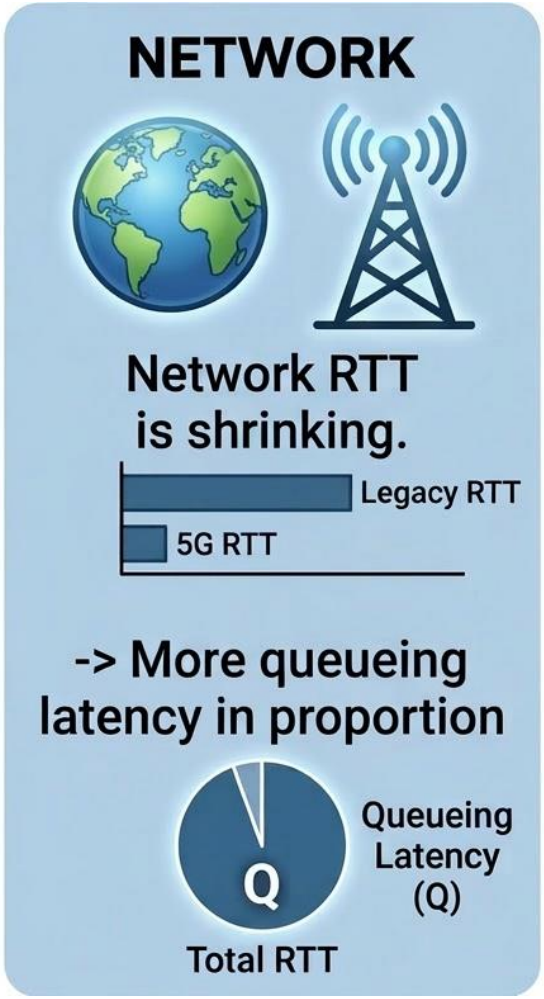
The burstiness is exacerbated by variations in **content complexity.**



Risk Overshooting!

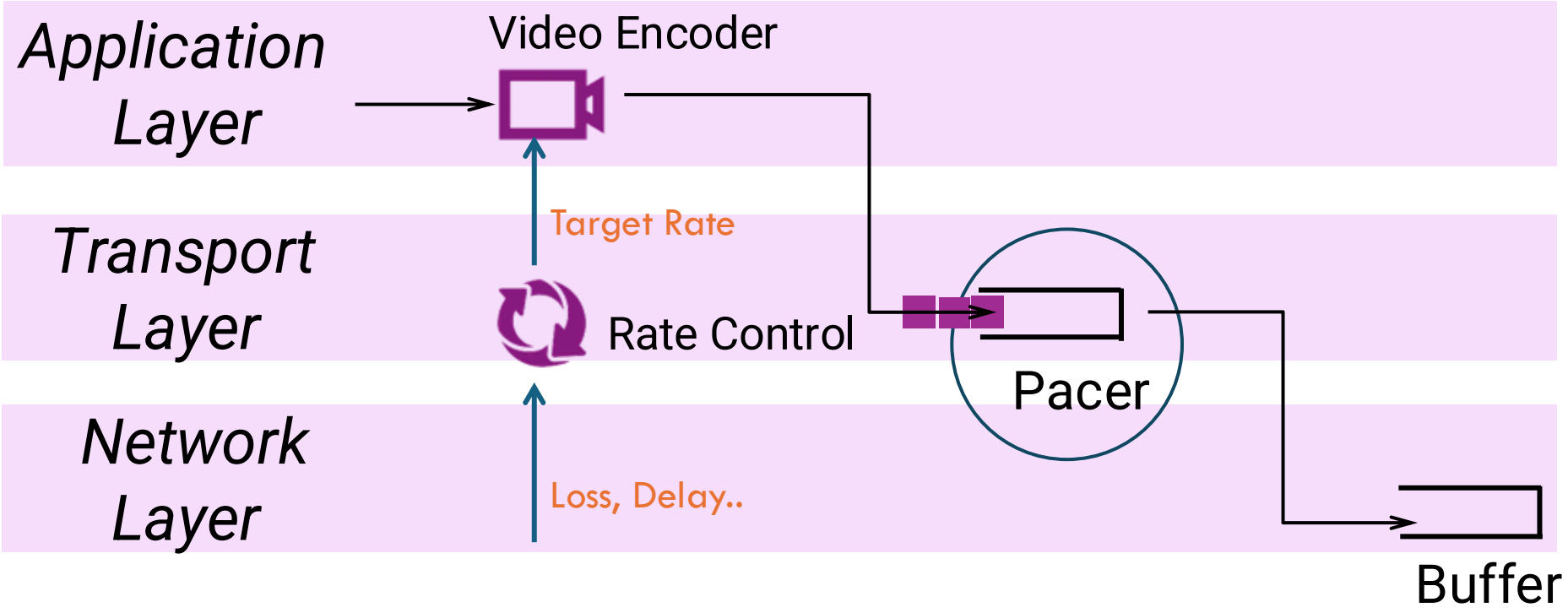


Trend
Why this matters more today



Motivation
Bursty Sending

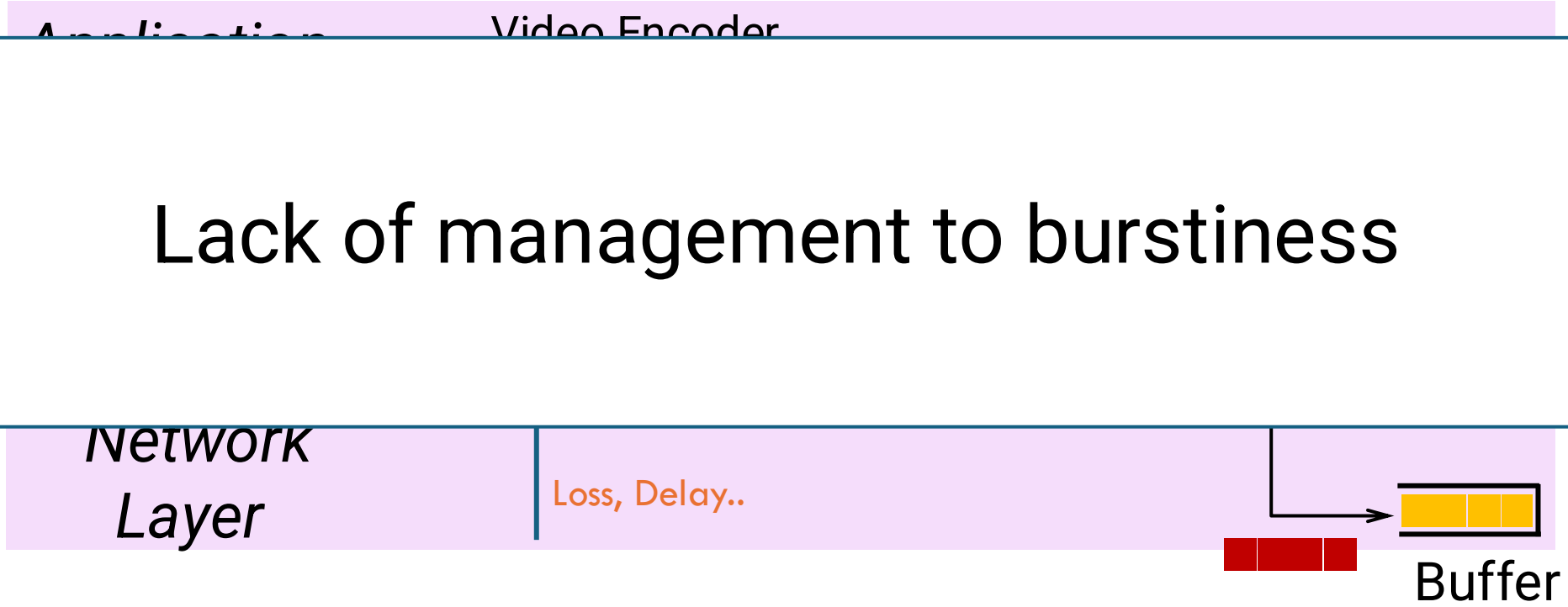
Can we just let go these bursty frames?



When pacer rate < network bandwidth: Better to let go bursts

Motivation
Bursty Sending

Can we just let go these bursty frames?



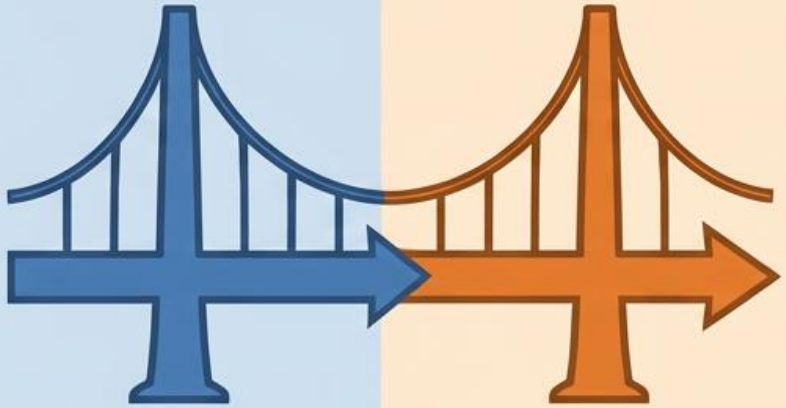
Lack of management to burstiness

However, bursts can also overshoots buffers.

Design ACE

Choices: **Managing burstiness** from both enqueue and dequeue perspectives.

ACE-N: Adapt the sending burstiness.

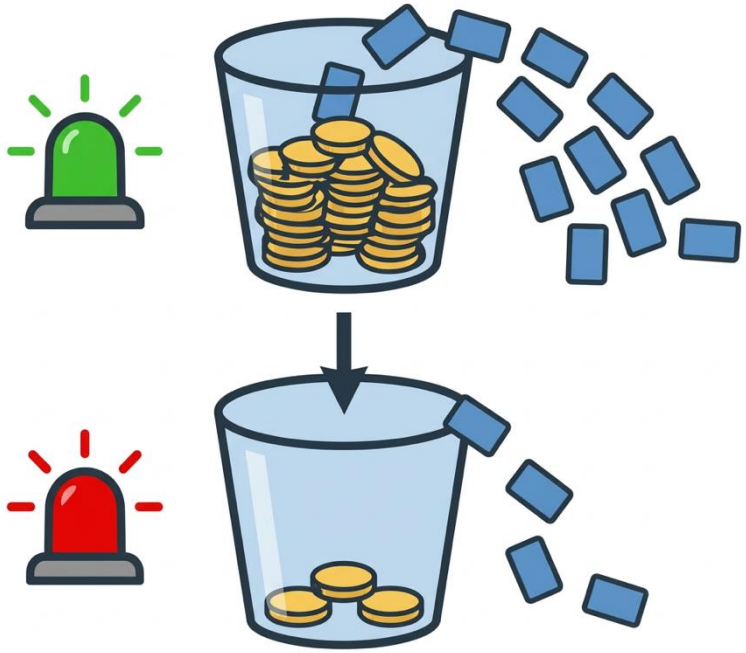


They Work Together

ACE-C: Adapt the encoding complexity.

ACE-N
Dequeue: Controlled sending pattern

Insight: Adaptively control the sending burstiness.



Network buffer is empty?

Burst more aggresively

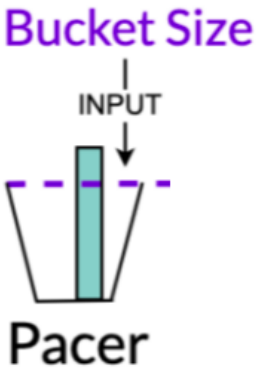
Risk overshooting buffer?

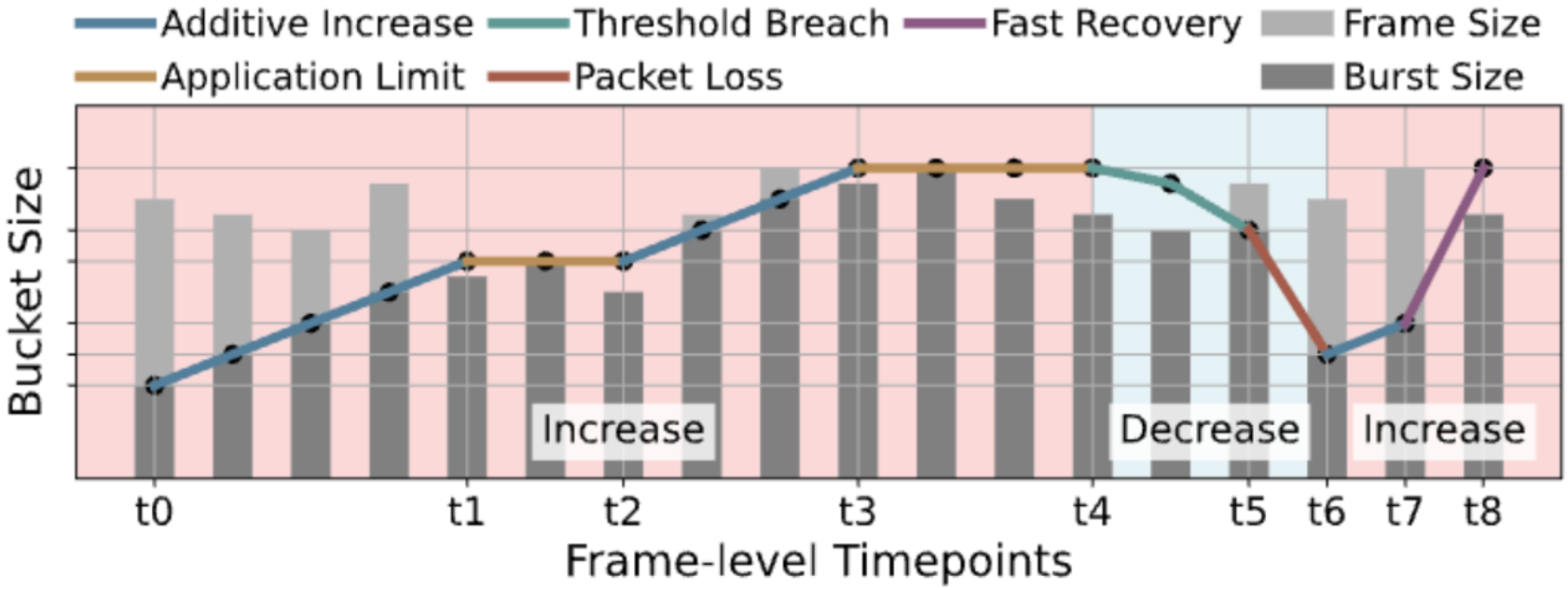
Shrink the bucket, pace more strictly.

ACE-N
Dequeue: Controlled sending pattern

We estimate the queuing size to adapt token bucket size.

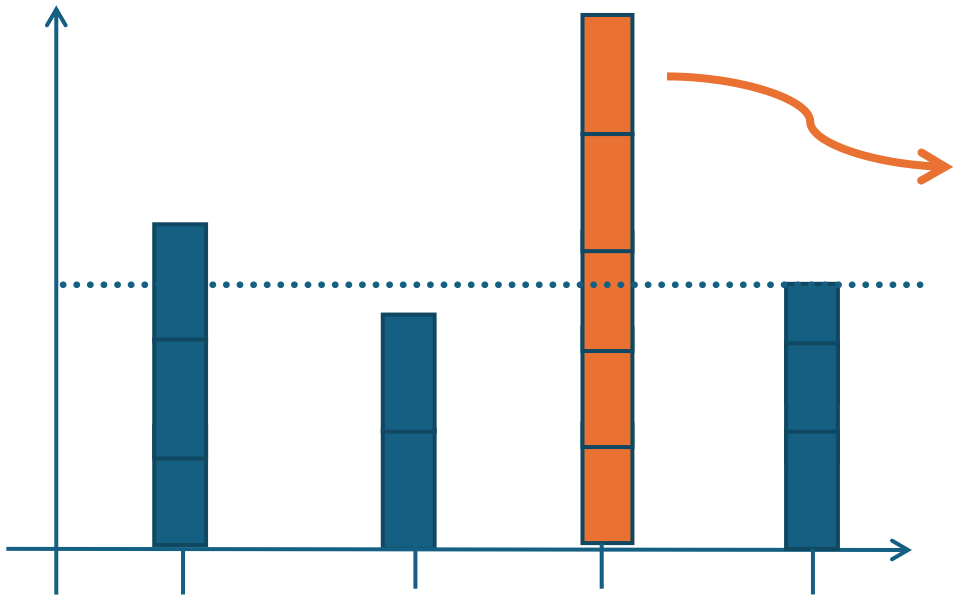
- Network queue size is estimated
- App Limited **Increase** to probe the bursty size.
- Loss and queue awared **Decrease** to avoid loss.





ACE-C
Enqueue: Smoother Frame Sizes

Encoder Side: Produce frames as smooth as possible.



Current Approach: Strict encoding rate control

More lossy compression

Worse quality

Evaluation shows strict rate control reduces VMAF quality for 10 ~ 15 points.

ACE-C
Enqueue: Smoother Frame Sizes

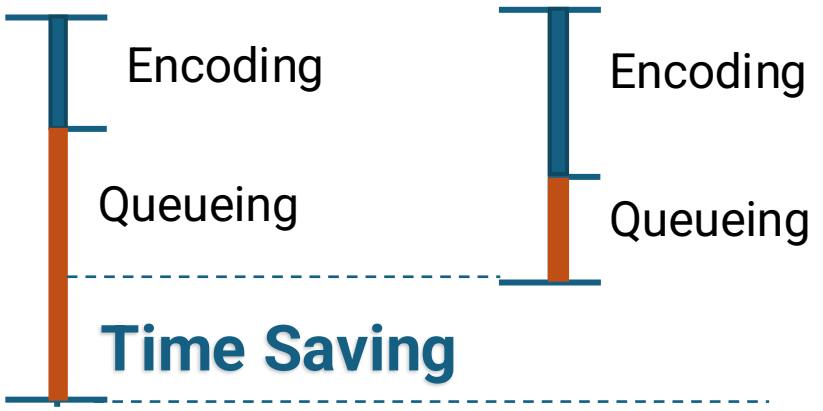
Insight: Adapting the encoding **complexity** to enable a smoother stream.

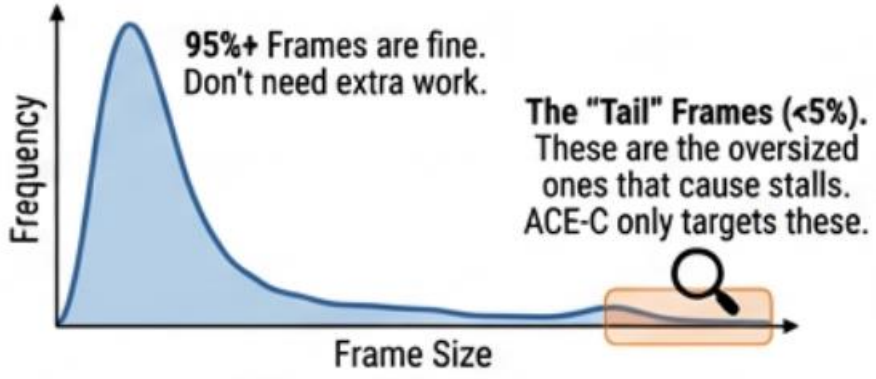
More advanced video coding technique

↑ Coding Complexity → ↑ Encoding Delay

↓ Frame Size → ↓ Pacing Delay

Trade-offs are possible:

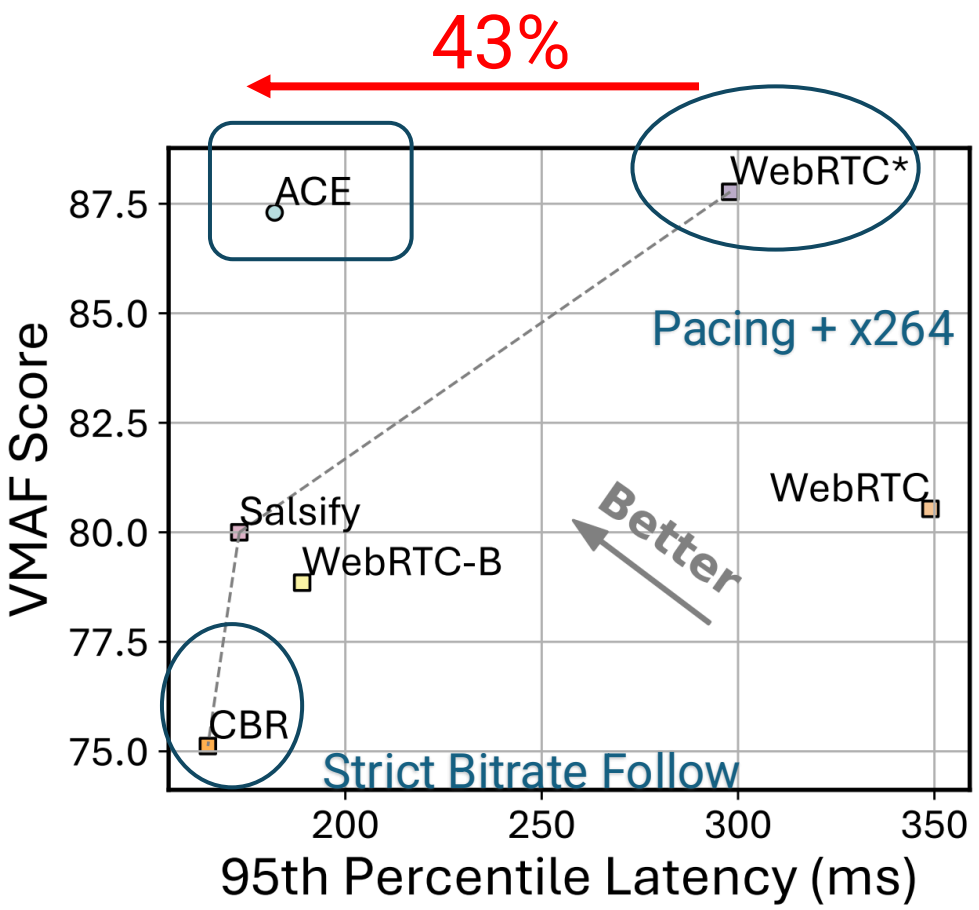




Evaluation
Main results

Emulation Setup:

- Mahimahi emulations with real world WIFI/ Cellular traces

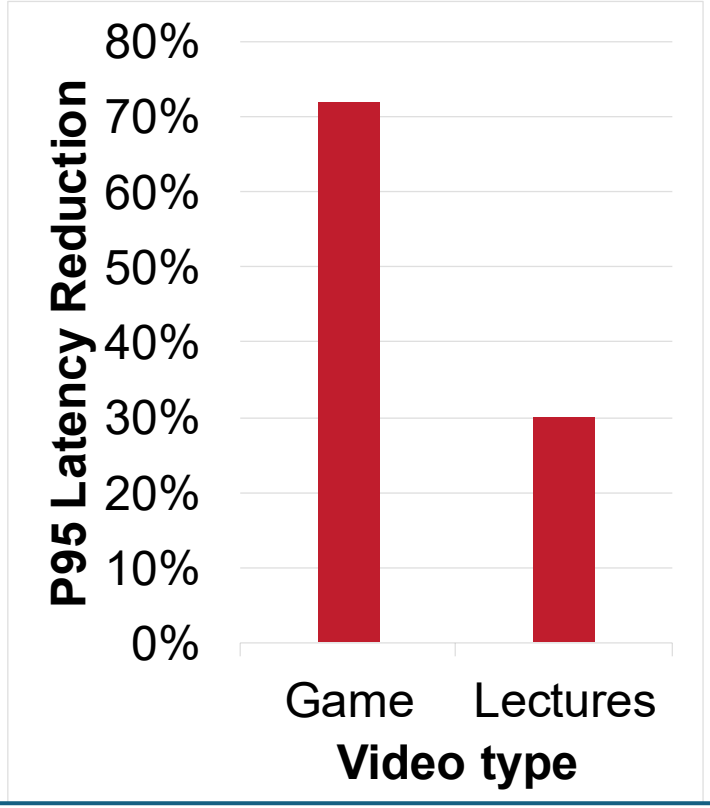


- Latency significantly reduced
- Quality remains high
- Superior to pacing/bursty/cbr baselines..

Experience
What we found in different environments

Sending burstiness control benifits most in Wi-Fi scenario and gaming videos.

Wi-Fi	4G	5G
43%	23%	35%



In cellular traces,the performance gains are less pronounced.

Cellular -> More fluctuation -> Congestion Control is more to be blame

Experience: Pay attention to burstiness management when **CC performs good** and **content is various**

Experience
From paper to real-world deployment

ACE-N is easy to deploy in existing systems, while ACE-C depends on having sufficient flexibility the codec.

Real-world Evaluation on campus Wi-Fi

WebRTC: Pacing controller

+

x264 Rate control & encoding parameter

Software encoding

ByteDance

Douyin Cloud Gaming

ACE-N deployed

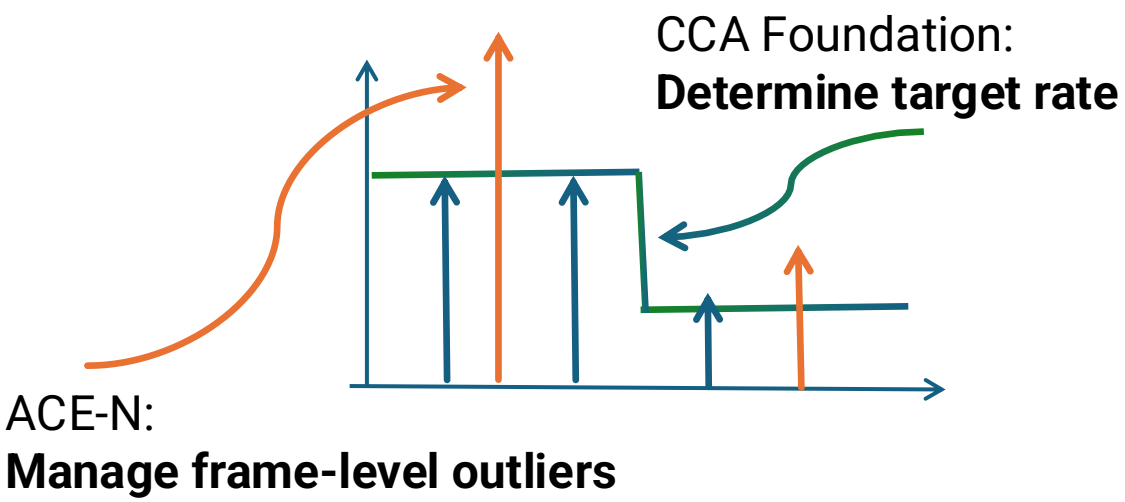
15%
Stall rate reduction

Limitation:

- Lack of control space in hardware codec
- Strict rate control preferred by real-time encoders

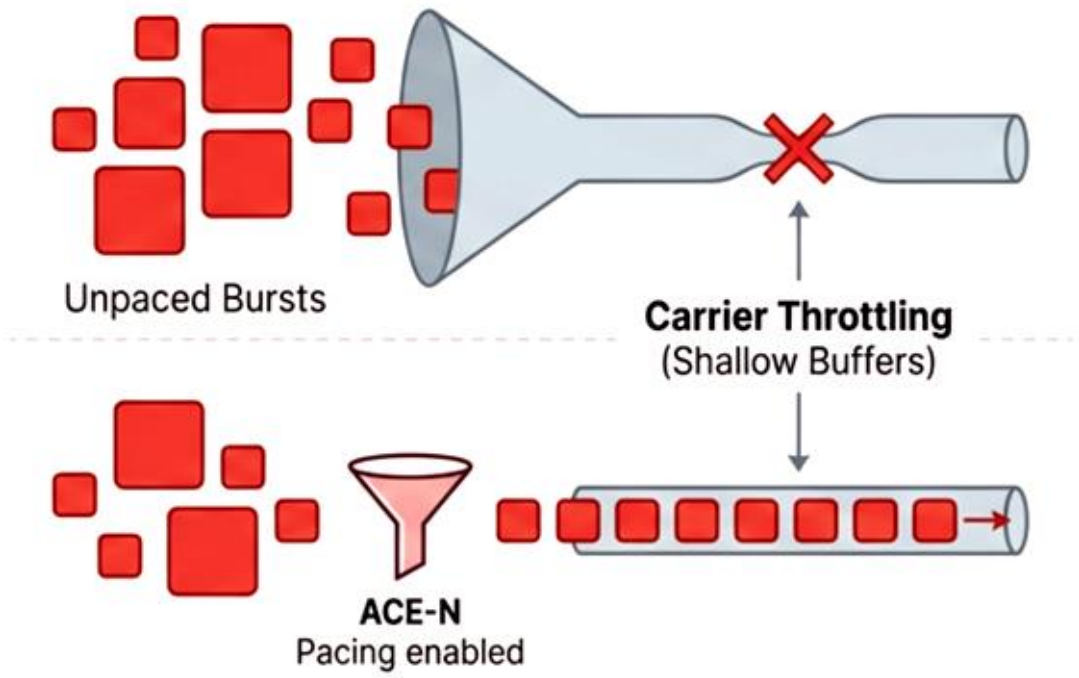
Experience
Lessons from ACE-N deployment

1. Clear boundary with CCA needed



Don't try to replace CCA

2. Carrier throttling requires pacing.



Shallow buffers by throttling allows ZERO bursts

What we learned

- In modern RTC, **sender-side pacing latency** can be a first-order problem.
- Average bitrate is not enough; **burstiness** matters.
- Strict pacing and blind bursting are both suboptimal.
- Cross-layer control is most useful when it **preserves quality** rather than trading quality away.

Thank you!

GitHub: SparkRTC